Summary in Graph

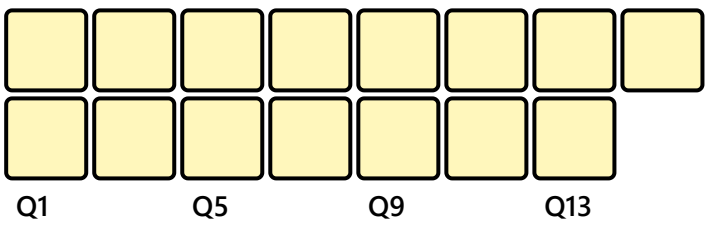
Exam Summary (GO Classes CS Test Series 2026 | Compiler Design | Topic Wise Test 1)

Qs. Attempted:	0 (0 + 0)	Correct Marks:	0 (0 + 0)
Correct Attempts:	0 (0 + 0)	Penalty Marks:	0 (0 + 0)
Incorrect Attempts:	0 (0 + 0)	Resultant Marks:	0 (0 + 0)

Total Questions:	15 (5 + 10)
Total Marks:	25 (5 + 20)
Exam Duration:	45 Minutes
Time Taken:	0 Minutes

Individual Question Statistics

Select Metric: Marks Obtained



* Average Stats based on first-attempt stats of Top 10% (136) out of 1358 users

EXAM RESPONSE EXAM STATS FEEDBACK

Technical

Q #1 Multiple Choice Type Award: 1 Penalty: 0.33 Compiler Design

Which of the following is true about the relationship between $\mathrm{LALR}(1)$ and $\mathrm{SLR}(1)$ parsers?

A. There are grammars that can be parsed by $\mathrm{SLR}(1)$ parsers that cannot be parsed by $\mathrm{LALR}(1)$ parsers.
B. There are grammars that can be parsed by $\mathrm{LALR}(1)$ parsers that cannot be parsed by $\mathrm{SLR}(1)$ parsers.
C. $\mathrm{LALR}(1)$ parsers are as powerful as $\mathrm{SLR}(1)$ parsers, so any grammar that can be parsed by an $\mathrm{LALR}(1)$ parser can be parsed by a $\mathrm{SLR}(1)$ parser.

D. For any grammar G which can be parsed by a $\mathrm{SLR}(1)$ parser, number of states in the $\mathrm{SLR}(1)$ parser can be less than number of states in the $\mathrm{LALR}(1)$ parser for G.

Your Answer: | Correct Answer: B | Not Attempted | Time taken: 00min 00sec | Discuss

Q #2 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Compiler Design

Which of the following is the name of the data structure in a compiler that is responsible for managing information about variables and their attributes?

A. Abstract Syntax Tree (AST)
B. Attribute Grammar
C. Symbol Table
D. Parse Table

Your Answer: | Correct Answer: C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #3 | Multiple Select Type | Award: 1 | Penalty: 0 | Compiler Design

Which of the following is true about grammar ambiguity?

A. A grammar is said to be ambiguous if it can produce more than one parse tree for some sentence/string.
B. A grammar is said to be ambiguous when, for some sentence, two different sequences of leftmost derivations can produce the same sentence from the same start symbol.
C. A grammar is ambiguous if there is some string w such that w's right-most derivation differs from its left-most derivation.
D. A grammar is said to be ambiguous if and only if, for some sentence, two different sequences of rightmost derivations can produce the same sentence from the same start symbol.

Your Answer: | Correct Answer: A;B;D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #4 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Compiler Design

In SLR parsing to get a shift-reduce conflict for state I on terminal symbol ' a ',

A. $\mathrm{A} \rightarrow \alpha . \beta$ with $\mathrm{First}\ (\beta)$ containing ' a ' should be in I
B. $\mathrm{A} \rightarrow \delta .$ be in I with $\mathrm{Follow}(\mathrm{A})$ having ' a '
C. $\mathrm{A} \rightarrow \alpha . a \beta$ should be in I and $\mathrm{X} \rightarrow \delta .$ be in I with $\mathrm{Follow}\ (\mathrm{X})$ having ' a '
D. None of the other options

Your Answer: | Correct Answer: C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #5 | Multiple Select Type | Award: 1 | Penalty: 0 | Compiler Design

Consider the following grammar:
$\mathrm{A} \rightarrow \mathrm{A}$ and $\mathrm{A} \mid \mathrm{A}$ or $\mathrm{A} \mid (\mathrm{A}) \mid$ true $\mid$ false
Which of the following is/are true about First sets for each production and nonterminal?

A. $\mathrm{FIRST}(\mathrm{true}) = \{$ "true" $\}$
B. $\mathrm{FIRST}($ false $) = \{$ "false" $\}$

C. $\text{FIRST}((\text{A}))=\{\text{“(”}\}$
D. $\text{FIRST}(\text{ A and A }) = \text{FIRST}(\text{A or A }) = \text{FIRST}(\text{A }) = \{\text{“true”, “false”}\}$

Your Answer: | Correct Answer: A;B;C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #6 Multiple Choice Type | Award: 2 | Penalty: 0.67 | Compiler Design

In LR parsing, If the parser is in a state containing an $\text{LR}(k)$ item $[\text{A} \rightarrow \alpha \bullet \gamma, \delta]$, what must be at the top of the stack?

A. α
B. γ
C. δ
D. A

Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Q #7 Multiple Choice Type | Award: 2 | Penalty: 0.67 | Compiler Design

In LR parsing, Given an $\text{LR}(k)$ item $[\text{A} \rightarrow \alpha \bullet \gamma, \delta]$, what lookahead allows a shift to be performed (ignoring any other $\text{LR}(k)$ items added to the state by closure)?

A. The terminal at the beginning of γ.
B. The terminal at the ending of α.
C. The terminal at the beginning of α.
D. The terminal at the beginning of δ.

Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Q #8 Multiple Choice Type | Award: 2 | Penalty: 0.67 | Compiler Design

In $\text{LR}(1)$ parsing, what constitutes a shift/reduce conflict?

A. A DFA state contains both LR items allowing shift and reduce for the different lookahead.
B. A DFA state contains both LR items allowing shift and reduce for the same lookahead.
C. A DFA state contains both LR items allowing shifts for the same lookahead.
D. A DFA state contains both LR items allowing shifts for the different lookahead.

Your Answer: | Correct Answer: B | Not Attempted | Time taken: 00min 00sec | Discuss

Q #9 Multiple Choice Type | Award: 2 | Penalty: 0.67 | Compiler Design

Consider the following sets of $\text{LR}(1)$ items in the states of a $\text{LR}(1)$ parser.

$$\begin{array}{ll} \text{State 0:} & \text{State 2:} \\ [\text{A} \rightarrow \bullet\text{a}, \text{b}] & [\text{A} \rightarrow \bullet\text{a}, \text{c}] \\ [\text{A} \rightarrow \text{a}\bullet, \text{c}] & [\text{A} \rightarrow \text{a}\bullet, \text{b}] \\ [\text{B} \rightarrow \text{a}\bullet, \text{b}] & [\text{B} \rightarrow \text{a}\bullet, \text{a}] \end{array}$$

$$\begin{array}{ll} \text{State 1:} & \text{State 3 :} \\ [\mathrm{A} \rightarrow \bullet \mathrm{a}, \mathrm{a}] & [\mathrm{A} \rightarrow \bullet \mathrm{a}, \mathrm{b}] \\ [\mathrm{A} \rightarrow \bullet \mathrm{a}, \mathrm{b}] & [\mathrm{B} \rightarrow \bullet \mathrm{a}, \mathrm{b}] \\ [\mathrm{B} \rightarrow \mathrm{a} \bullet, \mathrm{b}] & \end{array}$$

What states would be merged in a $\mathrm{LALR}(1)$ parser?

A. States 1 and 2.
B. States 0 and 1.
C. States 0 and 2.
D. States 2 and 3.

Your Answer: | Correct Answer: C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #10 Multiple Choice Type | Award: 2 | Penalty: 0.67 | Compiler Design

Consider the following sets of $\mathrm{LR}(1)$ items in the states of a $\mathrm{LR}(1)$ parser.

$$\begin{array}{ll} \text{State 0:} & \text{State 2:} \\ [\mathrm{A} \rightarrow \bullet \mathrm{a}, \mathrm{b}] & [\mathrm{A} \rightarrow \bullet \mathrm{a}, \mathrm{c}] \\ [\mathrm{A} \rightarrow \mathrm{a} \bullet, \mathrm{c}] & [\mathrm{A} \rightarrow \mathrm{a} \bullet, \mathrm{b}] \\ [\mathrm{B} \rightarrow \mathrm{a} \bullet, \mathrm{b}] & [\mathrm{B} \rightarrow \mathrm{a} \bullet, \mathrm{a}] \\ \text{State 1 :} & \text{State 3 :} \\ [\mathrm{A} \rightarrow \bullet \mathrm{a}, \mathrm{a}] & [\mathrm{A} \rightarrow \bullet \mathrm{a}, \mathrm{b}] \\ [\mathrm{A} \rightarrow \bullet \mathrm{a}, \mathrm{b}] & [\mathrm{B} \rightarrow \bullet \mathrm{a}, \mathrm{b}] \\ [\mathrm{B} \rightarrow \mathrm{a} \bullet, \mathrm{b}] & \end{array}$$

What happens when we merge states(which can be merged according to $\mathrm{LALR}(1)$ parser) in a $\mathrm{LALR}(1)$ parser?

A. A new reduce/reduce conflicts introduced in the $\mathrm{LALR}(1)$ parser
B. A new shift/reduce conflicts introduced in the $\mathrm{LALR}(1)$ parser
C. New reduce/reduce and shift/reduce conflicts introduced in the $\mathrm{LALR}(1)$ parser
D. No new conflicts introduced in the $\mathrm{LALR}(1)$ parser

Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Q #11 Multiple Choice Type | Award: 2 | Penalty: 0.67 | Compiler Design

Consider the following sets of $\mathrm{LR}(1)$ items in the states of a $\mathrm{LR}(1)$ parser for some grammar G.

$$\begin{array}{ll} \text{State 0:} & \text{State 2:} \\ [\mathrm{A} \rightarrow \bullet \mathrm{a}, \mathrm{b}] & [\mathrm{A} \rightarrow \bullet \mathrm{a}, \mathrm{c}] \\ [\mathrm{A} \rightarrow \mathrm{a} \bullet, \mathrm{c}] & [\mathrm{A} \rightarrow \mathrm{a} \bullet, \mathrm{b}] \\ [\mathrm{B} \rightarrow \mathrm{a} \bullet, \mathrm{b}] & [\mathrm{B} \rightarrow \mathrm{a} \bullet, \mathrm{a}] \end{array}$$

$$\begin{array}{ll} \text{State 1:} & \text{State 3 :} \\ [\mathrm{A} \rightarrow \bullet \mathrm{a}, \mathrm{a}] & [\mathrm{A} \rightarrow \bullet \mathrm{a}, \mathrm{b}] \\ [\mathrm{A} \rightarrow \bullet \mathrm{a}, \mathrm{b}] & [\mathrm{B} \rightarrow \bullet \mathrm{a}, \mathrm{b}] \\ [\mathrm{B} \rightarrow \mathrm{a} \bullet, \mathrm{b}] & \end{array}$$

Which of the following is true for the given set of $\mathrm{LR1}$ items and the given $\mathrm{LR1}$ parser (Assume the parser has only the given set of $\mathrm{LR1}$ items for G)?

A. G is LR1.
B. G is not LR1 because reduce/reduce conflict is caused by $[A \to a\bullet, c]$ and $[B \to a\bullet, b]$ in state 1.
C. G is not LR1 because Shift/reduce conflict is caused by $[A \to \bullet a, c]$ and $[B \to a\bullet, a]$ on lookahead a in state 2.
D. G is not LR1 because Shift/shift conflict is caused by $[A \to \bullet a, b]$ and $[B \to \bullet a, b]$ on lookahead a in state 3.

Your Answer: | Correct Answer: C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #12 | Numerical Type | Award: 2 | Penalty: 0 | Compiler Design

Consider the following grammar:

$$S \to S; a \mid a$$

Form the **augmented grammar** as: $S' \to S\$$

Determine the total number of states in the canonical collection of **SLR(1)** items for this augmented grammar, including the state that contains the accept production $\boxed{S' \to S\$\bullet}$.

Your Answer: | Correct Answer: 6 | Not Attempted | Time taken: 00min 00sec | Discuss

Q #13 | Numerical Type | Award: 2 | Penalty: 0 | Compiler Design

Consider the following grammar:

$$S \to abS \mid acS \mid c$$

Form the **augmented grammar** as:

$$S' \to S\$$$

Construct the **canonical collection of LR(0) items** for this augmented grammar. How many distinct states (item sets) are present in the canonical collection of LR(0) items, **excluding the state that contains the production** $\boxed{S' \to S\$\bullet}$ Count all other states.

Your Answer: | Correct Answer: 8 | Not Attempted | Time taken: 00min 03sec | Discuss

Q #14 | Multiple Select Type | Award: 2 | Penalty: 0 | Compiler Design

We have a scanner (lexical analyzer) "flex". Consider the following tokens and their associated regular expressions, given as a flex scanner specification :

Regular expression	Token
$(01 \mid 10)$	printf(" X ")
$0(01)^*1$	printf(" Y ")
$(1010^*1 \mid 0101^*0)$	printf(" Z ")

When using regular expressions to scan an input, we resolve conflicts by taking the largest possible match at any point. If two longest matches are of same length then we give preference in the below order
: $X > Y > Z$

For Which of the following inputs to this scanner, we will get a Lexical error?

A. 0101
B. 010101
C. 01010101
D. 0101010101

Your Answer: | Correct Answer: B;C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #15 | Multiple Select Type | Award: 2 | Penalty: 0 | Compiler Design

In lexical analysis, when using regular expressions to scan an input, for resolving conflicts that occur, our approach is usually to use "maximal-munch" (i.e. to always choose the longest possible match at any point), and if, even with maximal-munch we have multiple matching regular expressions, then we break ties based on the priorities of the regular expressions.

Consider the following scanner (lexical analyzer) “flex”.
Consider the following tokens and their associated regular expressions, given as a flex scanner specification :

Regular expression	Token
aa	print(“1”)
a	print(“2”)
ab	print(“3”)

If two longest matches are of same length then we give preference in the below order : $1 > 2 > 3$

For which of the following input strings: The string can be broken apart into substrings, where each substring matches one of the regular expressions, but • The maximal-munch algorithm will fail to break the string apart in a way where each piece matches one of the regular expressions.

A. $cabab$
B. aab
C. abb
D. aba

Your Answer: | Correct Answer: B | Not Attempted | Time taken: 00min 00sec | Discuss

Copyright & Stuff